Attack Methods, Perturbation Methods & Loss Functions

A comprehensive guide to all attack algorithms with mathematical equations.

Attack Methods Overview

Method	Config Value	Paper
PGD	`"pgd"`	Madry et al., 2017
BIM	`"bim"`	Kurakin et al., 2016
MI-FGSM	`"mim"`	Dong et al., 2017
Optim	`"optim"`	Optimization-based (Adam)

---

1 PGD (Projected Gradient Descent)

Paper: "Towards Deep Learning Models Resistant to Adversarial Attacks"

Core Idea

Iteratively perturb the input in the direction of the gradient sign, then project back to ε-ball.

Mathematical Formulation

Update Rule:

```
δ_{t+1} = Π_{ε} ( δ_t + α · sign(∇_δ L(f(x + δ), y)) )
```

Where:

- `δ` = adversarial perturbation (patch)
- `α` = step size (`STEP_LR` in config)
- `ε` = maximum perturbation bound (`EPSILON/255`)
- `Π_{ε}` = projection to ε-ball (clamp operation)
- `L` = loss function
- `f(x)` = detector output

Code Implementation

```
# attack/methods/pgd.py
update = self.step_lr * self.patch_obj.patch.grad.sign()
```

```
patch_tmp = self.patch_obj.patch + update
torch.clamp_(patch_tmp.data, min=0, max=ε)  # Projection
```

---

2 BIM (Basic Iterative Method)

Paper: "Adversarial examples in the physical world"

Core Idea

Same as PGD but with per-iteration clipping of the update magnitude.

Mathematical Formulation

Update Rule:

```
g_t = clip(α · sign(∇_δ L), -ε, ε)
δ_{t+1} = clip(δ_t + g_t, 0, 1)
```

Code Implementation

```
# attack/methods/bim.py
update = self.step_lr * grad.sign()
update = torch.clamp(update, min=-self.epsilon, max=self.epsilon)  # Clip update
patch_tmp = torch.clamp(self.patch + update, 0, 1)  # Clip final
```

---

3 MI-FGSM (Momentum Iterative FGSM)

Paper: "Boosting Adversarial Attacks with Momentum"

Core Idea

Accumulate gradients with momentum for better transferability and escaping local minima.

Mathematical Formulation

Momentum Update:

```
g_{t+1} = μ · g_t + ∇_δ L / ||∇_δ L||_1
```

Patch Update:

```
δ_{t+1} = clip(δ_t + α · sign(g_{t+1}), 0, 1)
```

Where:

- μ = momentum factor (default: 0.9)
- $||\cdot||_1$ = L1 norm

Code Implementation

```
# attack/methods/mim.py
now_grad = self.patch_obj.patch.grad
self.grad = self.grad * self.momentum + now_grad / torch.norm(now_grad, p=1)
update = self.step_lr * self.grad.sign()
```

---

4 Optim (Optimization-Based)

Core Idea

Use standard optimizers (Adam) instead of gradient-sign methods for smoother updates.

Mathematical Formulation (Adam)

First moment:

$$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$$

Second moment:

$$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$$

Update:

$$\delta_{t+1} = \delta_t - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \varepsilon)$$

Code Implementation

```
# train.py
optimizer = optim.Adam([patch], lr=args.lr, amsgrad=True)
loss.backward()
optimizer.step()
patch.clamp(0, 1)
```

🔧 Perturbation Methods (`PERTURB.GATE`)

Method	Description
`null`	No additional perturbation
`sharkdrop`	Dropout-style perturbation
`grad_descend`	Model weight perturbation

Note: These are model-level perturbations, not patch perturbations.

📉 Loss Functions

Main Loss Formula

```
L_total = L_detection + λ_tv · L_TV
```

---

1. Object Loss (`obj_loss`)

Purpose: Minimize detector's confidence on target class.

```
L_obj = mean(conf_scores)
```

Goal: Push confidence scores toward 0 → object "disappears"

---

2. Total Variation Loss (`tv_loss`)

Purpose: Encourage patch smoothness (printability).

```
L_TV = (1/N) · Σ |p_{i,j} - p_{i+1,j}| + |p_{i,j} - p_{i,j+1}|
```

Code:

```
# load_data.py TotalVariation class
tvcomp1 = Σ|patch[:,:,1:] - patch[:,:,:-1]|  # Horizontal
tvcomp2 = Σ|patch[:,1:,:] - patch[:,:-1,:]|  # Vertical
tv = (tvcomp1 + tvcomp2) / numel(patch)
```

---

3. MSE Losses (`ascend-mse`, `descend-mse`)

Descend MSE (minimize confidence):

```
L = MSE(conf, 0) = (1/N) · Σ(conf_i - 0)²
```

Ascend MSE (maximize confidence):

```
L = MSE(conf, 1) = (1/N) · Σ(conf_i - 1)²
```

---

4. Combined `obj-tv` Loss

```python
# utils/solver/loss.py
tv_loss = TVLoss.smooth(patch)
obj_loss = mean(confs)
return {'tv_loss': tv_loss, 'obj_loss': obj_loss}
```

Final loss:

```
L = obj_loss + tv_eta × tv_loss
```

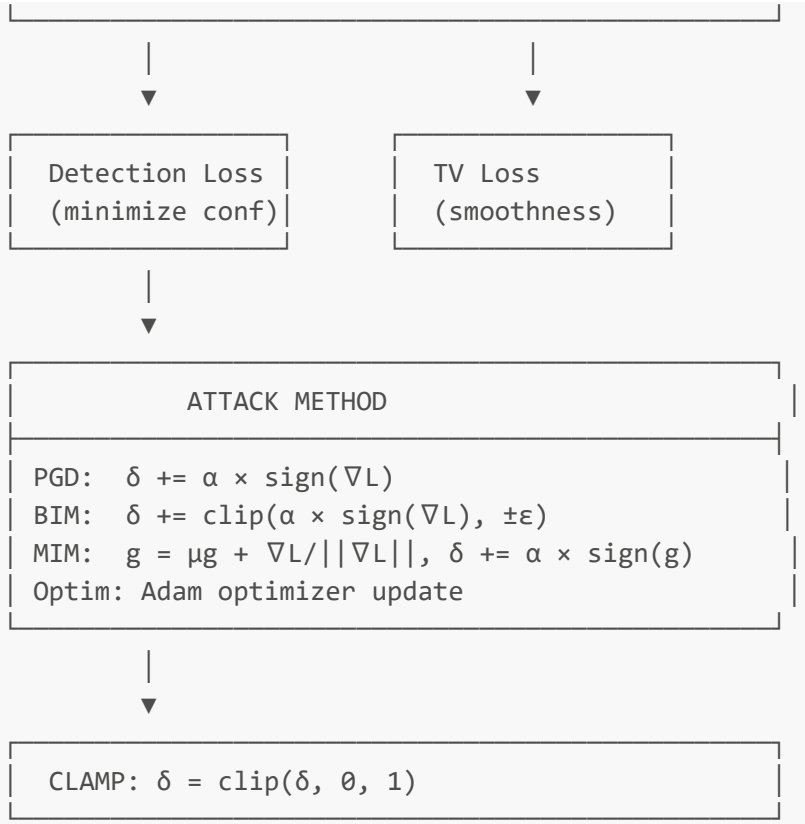
🎯 Detection Loss Types

Used in `MaxProbExtractor` classes:

Type	Formula	Description
`max_iou`	`score[argmax(IoU)]`	Score of highest IoU box
`max_conf`	`max(scores)`	Maximum confidence
`softplus_max`	`softplus(-log(1/s - 1))`	Smooth max confidence
`softplus_sum`	`Σ softplus(·) × IoU`	Weighted sum
`*_mtiou`	`score × IoU`	Multiply by IoU
`*_adiou`	`score + IoU`	Add IoU

📊 Summary Diagram

```
┌─────────────────────────────────────────────────────┐
│                  TOTAL LOSS                         │
│  L = L_det + λ_tv × L_TV                            │
```



Config Example

```yaml
ATTACKER:
  METHOD: "optim"      # pgd, bim, mim, optim
  STEP_LR: 0.03        # α (step size)
  EPSILON: 255         # ε × 255
  LOSS_FUNC: "obj-tv" # Loss function
  tv_eta: 2.5          # λ_tv
```